

CSES Flight Routes

1196 — As **k rotas mais baratas** e a variação de Dijkstra para os k menores caminhos

Algoritmos: **Dijkstra** + **Fila de Prioridade (min-heap)**

Integrantes: **Nícolas Queiroga** · **Mauricio Oliveira** · **João Lucas**



De “rotas de voo” a um dígrafo ponderado

O problema: encontrar os **k** caminhos mais baratos de Syrjälä (cidade 1) até Metsälä (cidade n) numa rede de voos de mão única.

Cidades



Vértices do grafo

Voo de mão única



Aresta direcionada

Preço do voo



Peso da aresta (não negativo)

k rotas mais baratas



k menores custos de caminho $1 \rightarrow n$

EXEMPLO PRÁTICO

Como a entrada vira a resposta

ENTRADA

```
4 6 3
1 2 1
1 3 3
2 3 2
2 4 6
3 2 8
3 4 1
```

SAÍDA

4 4 7

4

1ª e 2ª rotas — **empatam em 4** ($1 \rightarrow 3 \rightarrow 4 = 3+1$ e $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 = 1+2+1$).

7

3ª rota mais barata ($1 \rightarrow 2 \rightarrow 4 = 1+6$).

Rotas distintas de mesmo preço são contadas **separadamente** — por isso o 4 aparece duas vezes.

Dijkstra adaptado para k caminhos

- 1 Fila de prioridade mínima**
Guarda pares {custoAcumulado, vértice} e sempre entrega a rota pendente mais barata. Começa com (0, origem).
- 2 Extrair e relaxar**
A cada extração, estende a rota por todas as arestas de saída do vértice (relaxamento), empilhando os novos custos.
- 3 count[v] decide**
Cada vértice pode ser finalizado **até k vezes**. A i-ésima extração de v é o i-ésimo menor custo até v. Quando count[v] = k, v é descartado.
- 4 Parar na k-ésima extração do destino**
As k primeiras vezes que n sai da fila são exatamente as k rotas mais baratas — já em ordem crescente.

Por que a variação está correta

No Dijkstra clássico cada vértice é finalizado **uma vez**. Aqui permitimos **k finalizações** por vértice — e a corretude se apoia em três fatos.

Ordem crescente

A min-heap entrega os custos do menor para o maior. Com pesos ≥ 0 , nenhum custo extraído depois pode ser menor que um já extraído.

count substitui dist

Em vez do vetor `dist[]` único, `count[v]` conta extrações. A i -ésima saída de v fixa o i -ésimo menor custo até v .

Empates contam

Rotas diferentes de mesmo preço saem da fila separadamente — por isso valores repetidos (4 4) aparecem na resposta.

Limite k: processar v além de k vezes só geraria rotas piores que as k já garantidas — daí a poda `count[v] = k`.

O coração: min-heap + contagem de extrações

```
// count[v] = quantas vezes v ja saiu da fila; cada vertice e finalizado ate k vezes
pq.add(new long[]{0, 1}); // origem com custo 0
while (!pq.isEmpty() && achados < k) {
    long[] cur = pq.poll(); // menor custo pendente
    long d = cur[0]; int u = (int) cur[1];
    if (count[u] >= k) continue; // ja temos k custos para u
    count[u]++;
    if (u == n) { sb.append(d).append(' '); achados++; }
    for (long[] e : adj[u]) // relaxamento das arestas de saida
        if (count[(int) e[0]] < k) pq.add(new long[]{d + e[1], e[0]});
}
```

Custo computacional e cuidados

$$O(k \cdot E)$$

Inserções na fila ($\leq k$ finalizações $\times$ arestas)

$$O(\log k \cdot E)$$

Custo de cada operação da min-heap

$$O(k \cdot E \cdot \log k \cdot E)$$

Tempo total $\cdot$ memória $O(V + E) + O(k \cdot E)$

Com $E \leq 2 \cdot 10^5$ e $k \leq 10 \rightarrow \sim 2 \cdot 10^6$ operações; cabe no limite de 1 s do CSES (maior teste aceito $\sim 0,70$ s).

- **Rotas de mesmo preço** $\rightarrow$ contadas individualmente (ex.: 4 4 no exemplo).
- **Cidades repetidas na rota** $\rightarrow$ permitidas; não restringimos a caminhos simples.
- **Custos grandes** $\rightarrow$ uso de `long` evita overflow (respostas como 4191432395).
- **Entrada grande** ($2 \cdot 10^5$ arestas) $\rightarrow$ leitura por buffer de bytes evita TLE.

Exemplo: $n=4, m=6, k=3 \rightarrow 4 \ 4 \ 7 \cdot$ **Accepted no CSES** (17/17 testes)